

```
!pip install ISLP
from ISLP import load_data
import seaborn as sns
import matplotlib.pyplot as plt
auto = load_data('Auto')
```

Requirement already satisfied: ISLP in /usr/local/lib/python3.12/dist-packages (0.4.0)

Requirement already satisfied: numpy>=1.7.1 in /usr/local/lib/python3.12/dist-packages (from ISLP) (2.0.2)

Requirement already satisfied: scipy>=0.9 in /usr/local/lib/python3.12/dist-packages (from ISLP) (1.16.3)

Requirement already satisfied: pandas>=0.20 in /usr/local/lib/python3.12/dist-packages (from ISLP) (2.2.2)

Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (from ISLP) (5.4.0)

Requirement already satisfied: scikit-learn>=1.2 in /usr/local/lib/python3.12/dist-packages (from ISLP) (1.6.1)

Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from ISLP) (1.5.2)

Requirement already satisfied: statsmodels>=0.13 in /usr/local/lib/python3.12/dist-packages (from ISLP) (0.14.5)

Requirement already satisfied: lifelines in /usr/local/lib/python3.12/dist-packages (from ISLP) (0.30.0)

Requirement already satisfied: pygam in /usr/local/lib/python3.12/dist-packages (from ISLP) (0.10.1)

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from ISLP) (2.8.0+cu126)

Requirement already satisfied: pytorch-lightning in /usr/local/lib/python3.12/dist-packages (from ISLP) (2.5.6)

Requirement already satisfied: torchmetrics in /usr/local/lib/python3.12/dist-packages (from ISLP) (1.8.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.20->ISLP) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.20->ISLP) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.20->ISLP) (2025.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.2->ISLP) (3.6.0)

Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.13->ISLP) (1.0.2)

Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.13->ISLP) (25.0)

Requirement already satisfied: matplotlib>=3.0 in /usr/local/lib/python3.12/dist-packages (from lifelines->ISLP) (3.10.0)

Requirement already satisfied: autograd>=1.5 in /usr/local/lib/python3.12/dist-packages (from lifelines->ISLP) (1.8.0)

Requirement already satisfied: autograd-gamma>=0.3 in /usr/local/lib/python3.12/dist-packages (from lifelines->ISLP) (0.5.0)

Requirement already satisfied: formulaic>=0.2.2 in /usr/local/lib/python3.12/dist-packages (from lifelines->ISLP) (1.2.1)

Requirement already satisfied: progressbar2<5,>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from pygam->ISLP) (4.5.0)

Requirement already satisfied: tqdm>=4.57.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning->ISLP) (4.67.1)

Requirement already satisfied: PyYAML>5.4 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning->ISLP) (6.0.3)

Requirement already satisfied: fsspec>=2022.5.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (2025.3.0)

Requirement already satisfied: typing-extensions>4.5.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning->ISLP) (4.15.0)

Requirement already satisfied: lightning-utilities>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning->ISLP) (0.15.2)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (3.20.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (1.13.3)

Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (3.5)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (3.1.6)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (11.7.1.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (2.27.3)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (12.6.85)

Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (1.11.1.6)

Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch->ISLP) (3.4.0)

Requirement already satisfied: interface-meta>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines->ISLP) (1.3.0)

Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines->ISLP) (2.10.2)

Requirement already satisfied: wrapt>=1.0 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines->ISLP) (2.0.0)

Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (3.13.2)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (1.4.9)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines->ISLP) (3.2.5)

Requirement already satisfied: python-utils>=3.8.1 in /usr/local/lib/python3.12/dist-packages (from progressbar2<5,>=4.2.0->pygam->ISLP) (3.9.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=0.20->ISLP) (1.17.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->ISLP) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch->ISLP) (3.0.3)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (2.6.1)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (25.4.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (6.7.0)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (0.4.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (1.22.0)

Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-packages (from yarl<2.0,>=1.17.0->aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch-lightning->ISLP) (3.11)

14. In this problem, you will develop a model to predict whether a given car gets high or low gas mileage based on the Auto data set.

(a) Create a binary variable, mpg01, that contains a 1 if mpg contains a value above its median, and a 0 if mpg contains a value below its median. You can compute the median using the median() method of the data frame. Note you may find it helpful to add a column mpg01 to the data frame by assignment. Assuming you have stored the data frame as Auto, this can be done as follows: Auto['mpg01'] = mpg01

```
auto.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
name								
chevrolet chevelle malibu	18.0	8	307.0	130	3504	12.0	70	1
buick skylark 320	15.0	8	350.0	165	3693	11.5	70	1
plymouth satellite	18.0	8	318.0	150	3436	11.0	70	1
amc rebel sst	16.0	8	304.0	150	3433	12.0	70	1
ford torino	17.0	8	302.0	140	3449	10.5	70	1

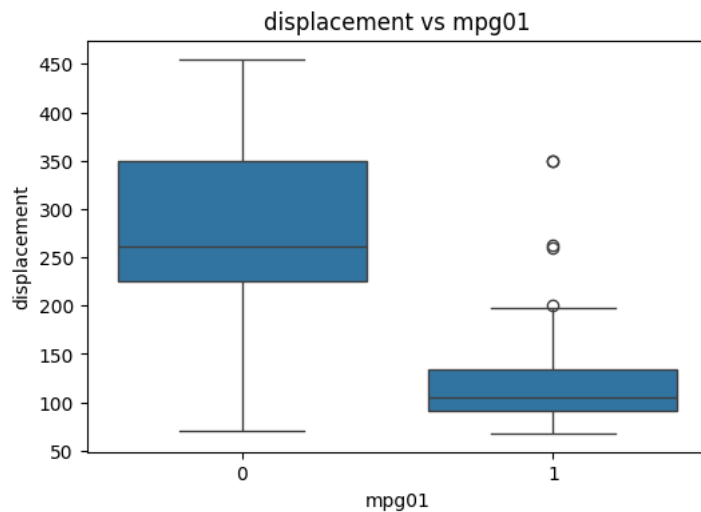
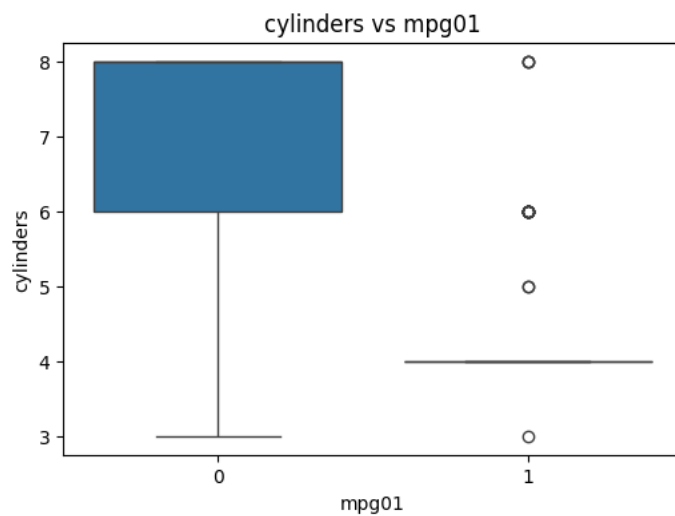
```
mpg_median = auto['mpg'].median()
auto['mpg01'] = (auto['mpg'] > mpg_median).astype(int)
print("Median mpg:", mpg_median)
print(auto[['mpg', 'mpg01']].head())
```

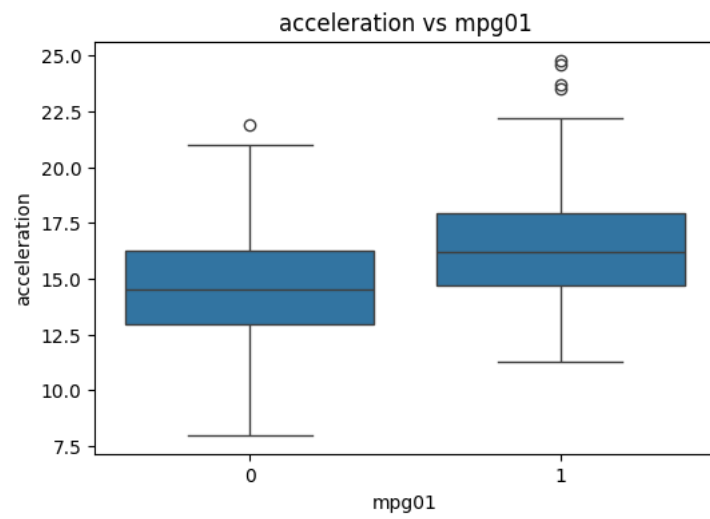
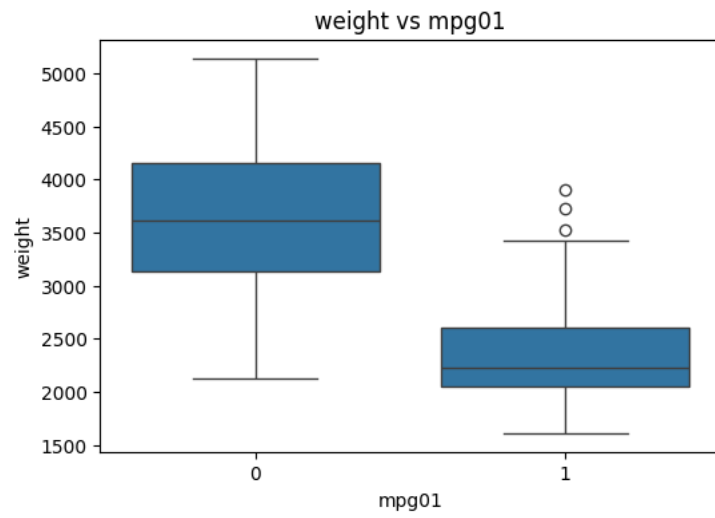
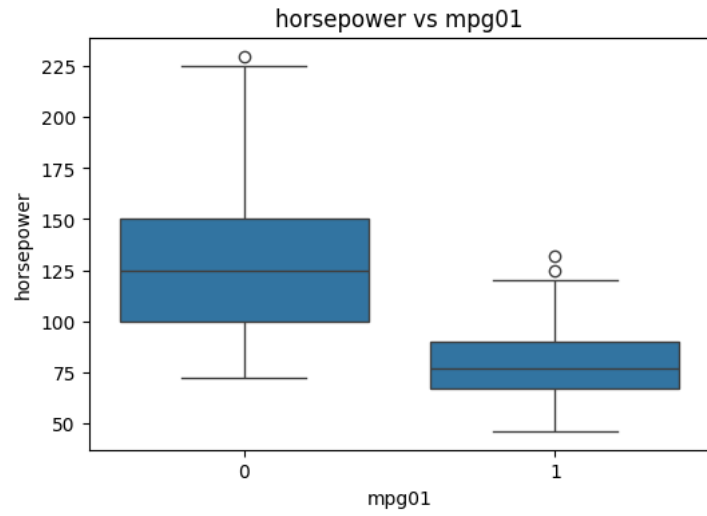
Median mpg: 22.75

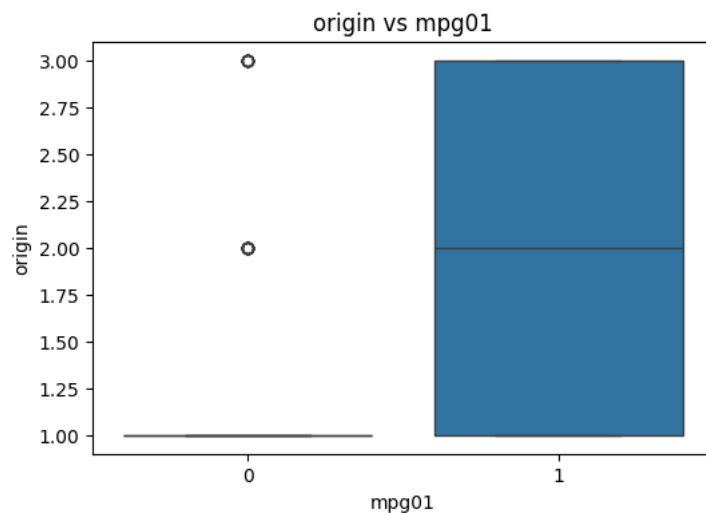
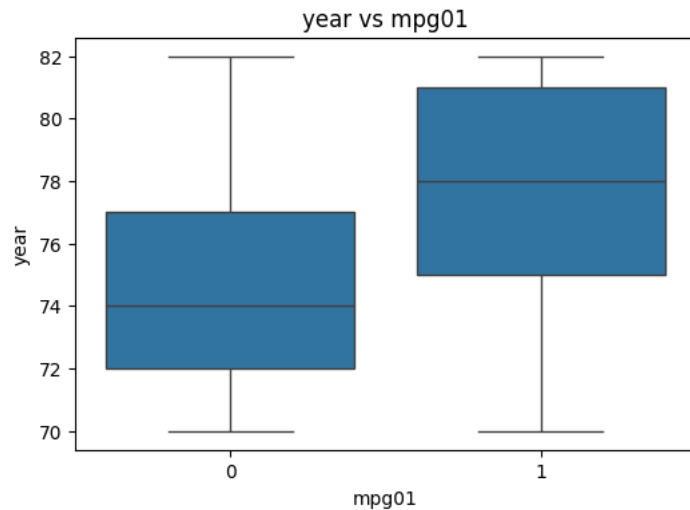
	mpg	mpg01
name		
chevrolet chevelle malibu	18.0	0
buick skylark 320	15.0	0
plymouth satellite	18.0	0
amc rebel sst	16.0	0
ford torino	17.0	0

(b) Explore the data graphically in order to investigate the association between mpg01 and the other features. Which of the other features seem most likely to be useful in predicting mpg01? Scatterplots and boxplots may be useful tools to answer this question. Describe your findings.

```
features = ['cylinders', 'displacement', 'horsepower', 'weight',  
'acceleration', 'year', 'origin']  
for col in features:  
    plt.figure(figsize=(6, 4))  
    sns.boxplot(x='mpg01', y=col, data=auto)  
    plt.title(f'{col} vs mpg01')  
    plt.show()
```







If the two boxes for $\text{mpg01} = 0$ and $\text{mpg01} = 1$ differ greatly or barely overlap, it means that variable is strongly associated with gas mileage.

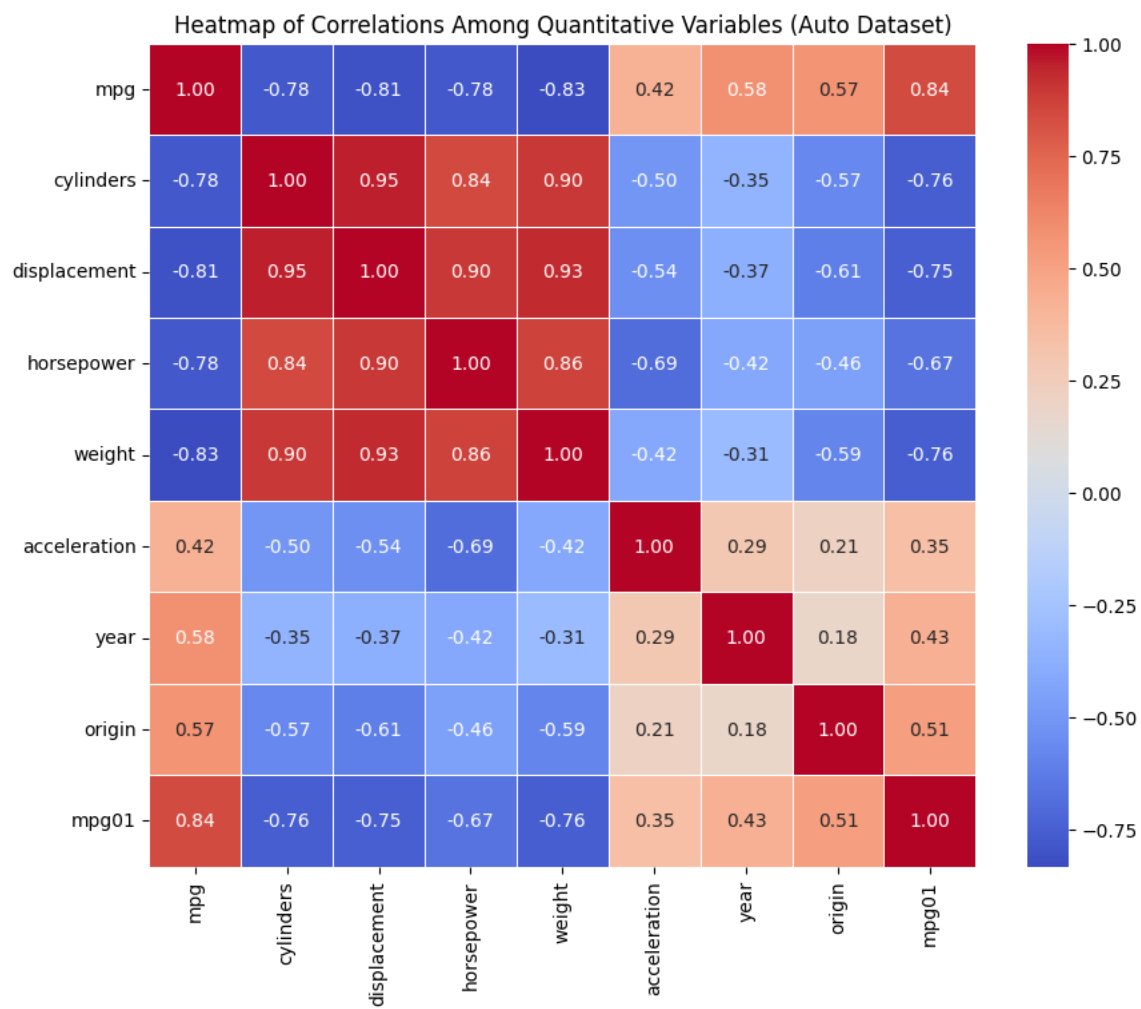
- Cars with high gas mileage ($\text{mpg01} = 1$) are generally lighter. → Heavier cars tend to have lower fuel efficiency.
- Cars with $\text{mpg01} = 1$ usually have lower horsepower. → High horsepower cars often have worse mileage.
- Cars with smaller engine displacement or fewer cylinders are more likely to have $\text{mpg01} = 1$.
→ Smaller engines are more fuel-efficient.

```
auto_num = auto.select_dtypes(include=['number'])

# Compute correlation matrix
corr_matrix = auto_num.corr()

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
            linewidths=0.5)
```

```
plt.title('Heatmap of Correlations Among Quantitative Variables (Auto Dataset)')
plt.show()
```



mpg01 is negatively correlated with weight, displacement, cylinders, and horsepower

```
sns.pairplot(auto, vars=['mpg', 'displacement', 'horsepower',
'weight', 'year'], hue='mpg01')
plt.show()
```



(c) Split the data into a training set and a test set.

```
from sklearn.model_selection import train_test_split
train, test = train_test_split(auto, test_size=0.3, random_state=1)
```

(d) Perform LDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix
import pandas as pd

predictors = ['cylinders', 'displacement', 'horsepower', 'weight']
```

```

# Train LDA model
lda = LinearDiscriminantAnalysis()
lda.fit(train[predictors], train['mpg01'])
lda_pred = lda.predict(test[predictors])

cm = confusion_matrix(test['mpg01'], lda_pred)
acc = accuracy_score(test['mpg01'], lda_pred)

cm_df = pd.DataFrame(
    cm,
    index=["Actual 0", "Actual 1"],
    columns=["Predicted 0", "Predicted 1"]
)

print("Confusion Matrix:\n", cm_df)
print("\nTest Accuracy:", round(acc, 3))
print("Test Error Rate:", round(1 - acc, 3))

```

```

Confusion Matrix:

```

	Predicted 0	Predicted 1
Actual 0	58	7
Actual 1	3	50

```

Test Accuracy: 0.915
Test Error Rate: 0.085

```

(e) Perform QDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```

from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
qda = QuadraticDiscriminantAnalysis()
qda.fit(train[predictors], train['mpg01'])

```

```

# 在測試集上預測

```

```

qda_pred = qda.predict(test[predictors])
qda_prob = qda.predict_proba(test[predictors]) # 若要看機率

```

```

# 計算混淆矩陣、準確率與錯誤率

```

```

cm = confusion_matrix(test['mpg01'], qda_pred)
accuracy = accuracy_score(test['mpg01'], qda_pred)
test_error = 1 - accuracy

```

```

cm_df = pd.DataFrame(
    cm,
    index=["Actual 0", "Actual 1"],
    columns=["Predicted 0", "Predicted 1"]
)

```

```

print(cm_df)
print(f"\nAccuracy: {accuracy:.4f}")
print(f"Test error rate: {test_error:.4f}")

```

	Predicted 0	Predicted 1
Actual 0	61	4
Actual 1	5	48

Accuracy: 0.9237

Test error rate: 0.0763

(f) Perform logistic regression on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(train[predictors], train['mpg01'])

y_pred = log_model.predict(test[predictors])

# 計算準確率與錯誤率
acc = accuracy_score(test['mpg01'], y_pred)
test_error = 1 - acc
cm = confusion_matrix(test['mpg01'], y_pred)
cm_df = pd.DataFrame(
    cm,
    index=["Actual 0", "Actual 1"],
    columns=["Predicted 0", "Predicted 1"]
)

print(cm_df)
print(f"Test Accuracy: {acc:.3f}")
print(f"Test Error Rate: {test_error:.3f}")
```

	Predicted 0	Predicted 1
Actual 0	59	6
Actual 1	4	49

Test Accuracy: 0.915
Test Error Rate: 0.085

(g) Perform naive Bayes on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
from sklearn.naive_bayes import GaussianNB
nb_model = GaussianNB()
nb_model.fit(train[predictors], train['mpg01'])

bayes_pred = nb_model.predict(test[predictors])

# 計算準確率與錯誤率
acc = accuracy_score(test['mpg01'], bayes_pred)
test_error = 1 - acc
cm = confusion_matrix(test['mpg01'], bayes_pred)
```

```

cm_df = pd.DataFrame(
    cm,
    index=["Actual 0", "Actual 1"],
    columns=["Predicted 0", "Predicted 1"]
)

print(cm_df)
print(f"Test Accuracy: {acc:.3f}")
print(f"Test Error Rate: {test_error:.3f}")

```

```

          Predicted 0  Predicted 1
Actual 0             60           5
Actual 1             5          48
Test Accuracy: 0.915
Test Error Rate: 0.085

```

(h) Perform KNN on the training data, with several values of K, in order to predict mpg01. Use only the variables that seemed most associated with mpg01 in (b). What test errors do you obtain? Which value of K seems to perform the best on this data set?

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
train_scaled = scaler.fit_transform(train[predictors])
test_scaled = scaler.transform(test[predictors])

# 嘗試多個 K 值
k_values = range(1,21)
test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(train_scaled, train['mpg01'])
    y_pred = knn.predict(test_scaled)
    acc = accuracy_score(test['mpg01'], y_pred)
    test_error = 1 - acc
    test_errors.append(test_error)
    print(f"K = {k}, Test Error = {test_error:.3f}")

# 找出最佳 K
best_k = k_values[test_errors.index(min(test_errors))]
print(f"\nBest K: {best_k} (Test Error = {min(test_errors):.3f})")

K = 1, Test Error = 0.119
K = 2, Test Error = 0.085
K = 3, Test Error = 0.102
K = 4, Test Error = 0.076
K = 5, Test Error = 0.085
K = 6, Test Error = 0.085
K = 7, Test Error = 0.076
K = 8, Test Error = 0.059
K = 9, Test Error = 0.076

```

K = 10, Test Error = 0.085
K = 11, Test Error = 0.076
K = 12, Test Error = 0.085
K = 13, Test Error = 0.085
K = 14, Test Error = 0.085
K = 15, Test Error = 0.085
K = 16, Test Error = 0.085
K = 17, Test Error = 0.085
K = 18, Test Error = 0.085
K = 19, Test Error = 0.085
K = 20, Test Error = 0.085

Best K: 8 (Test Error = 0.059)